AFL 模糊测试实验

实验环境：Ubuntu 18.04 64 位虚拟机

实验工具：原版 AFL、AFL-Qemu、coreutils-9.1

测试目标程序：cat、wc、cut

实验日期：2026-06-06

实验人员：魏朝旭

学号：23009201082

目录

1. 实验目的

2. 实验环境与工具

3. 实验准备

4. 实验一：基于 afl-gcc 静态编译插桩模糊测试

5. 实验二：基于 AFL-Qemu 动态插桩模糊测试

6. 输入种子对 Fuzz 结果的影响分析

7. 静态插桩与 Qemu 动态插桩原理及效果差异分析

8. 实验总结

1. 实验目的

1. 掌握 AFL 模糊测试工具的安装、配置与基础使用方法。

2. 理解 AFL 静态编译插桩与 QEMU 动态二进制插桩两种工作模式的原理。

3. 学会构造不同类型输入种子，对比种子对模糊测试路径覆盖率、执行效率的影响。

4. 完成对 Coreutils 系统工具集的模糊测试，分析两类插桩模式的优缺点与适用场景。

2. 实验环境与工具

2.1 硬件环境

虚拟机：Ubuntu 18.04 64bit，多核 CPU，内存配置满足编译与测试需求。

2.2 软件工具

1. AFL：模糊测试框架，版本 2.57b

2. AFL-Qemu：AFL 二进制动态插桩组件

3. coreutils-9.1：GNU 系统工具集，本次测试目标程序

4. GCC、afl-gcc：程序编译工具链

3. 实验准备

3.1 安装系统编译依赖

打开终端，执行命令安装编译 AFL、QEMU、Coreutils 所需依赖包：

```
bash
sudo apt update
sudo apt install build-essential libtool libtool-bin automake bison flex libglib2.0-dev
zlib1g-dev libpixman-1-dev
```

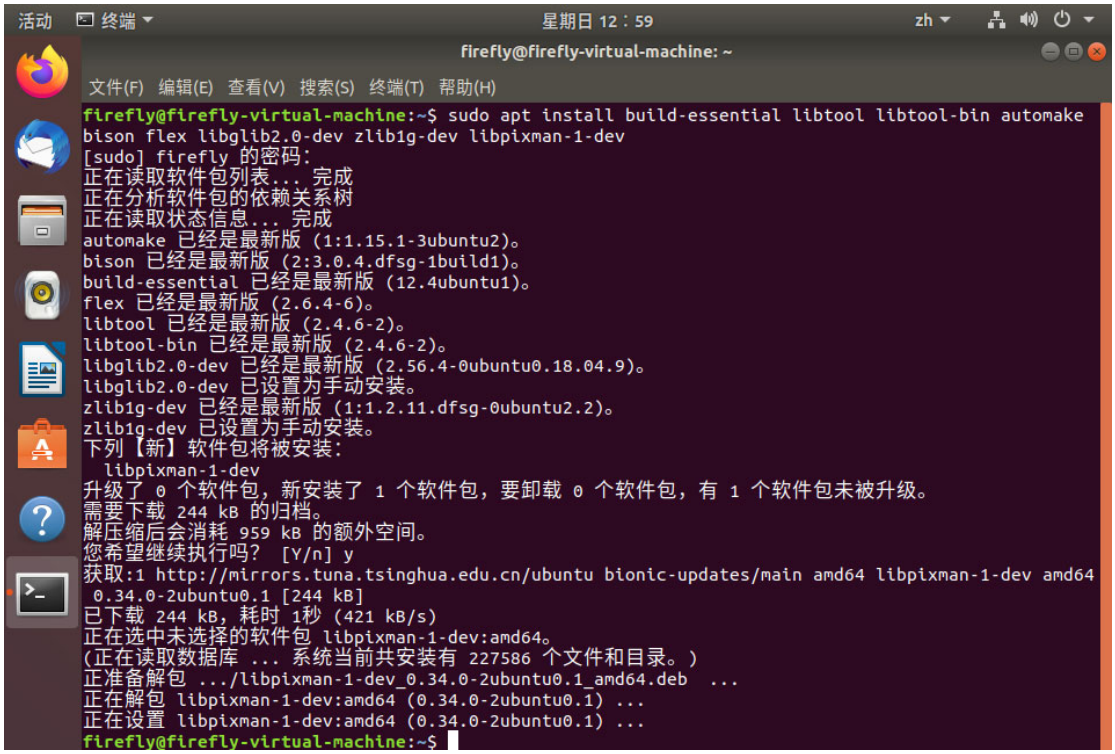


图 1：依赖包安装执行界面及完成提示

3.2 安装原版 AFL

克隆 AFL 官方源码并编译安装：

```bash
git clone https://github.com/google/AFL.git
cd AFL
make
sudo make install
```

验证工具安装成功：

```bash
afl-gcc -v
afl-fuzz -h
```

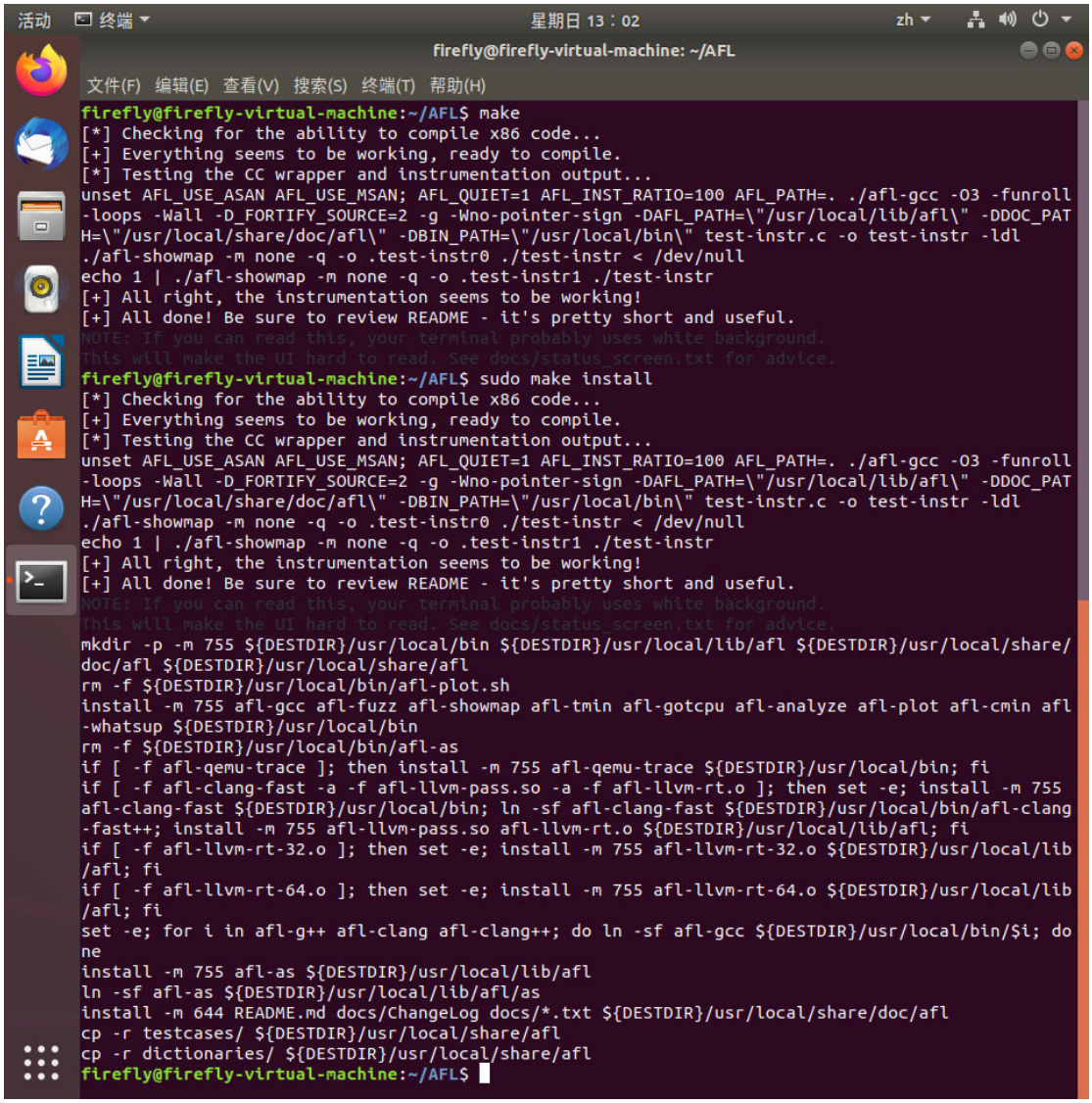


图 2：make 编译 AFL 过程

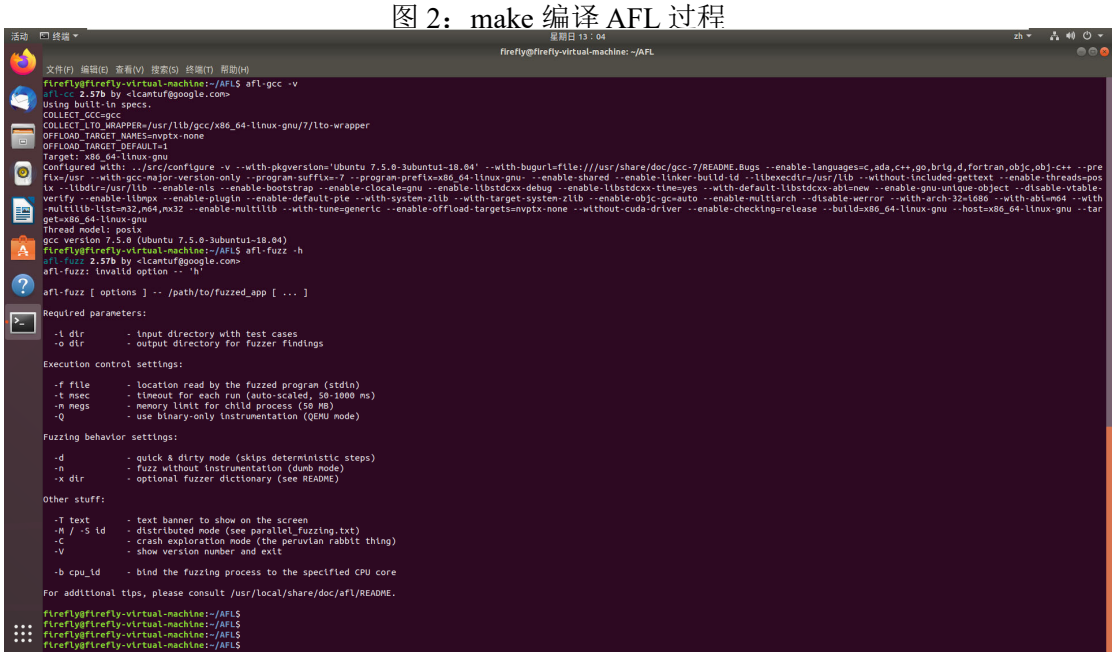


图 3：afl-gcc -v、afl-fuzz -h 执行结果，确认工具正常可用

3.3 在线编译部署 AFL-Qemu

进入 AFL 内置的 qemu_mode 目录，使用官方脚本在线编译 QEMU：

```bash
cd ~/AFL/qemu_mode
./build_qemu_support.sh
```

编译完成后验证 QEMU 模式：

```bash
afl-fuzz -h
```

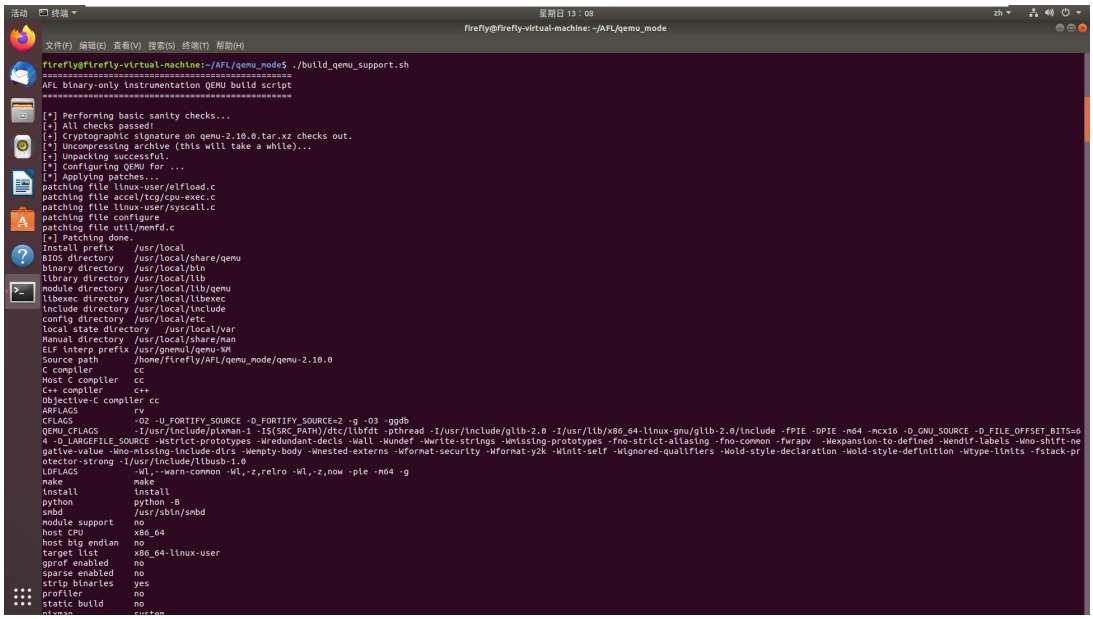


图 4：build_qemu_support.sh 脚本执行、在线下载并编译 QEMU 全过程

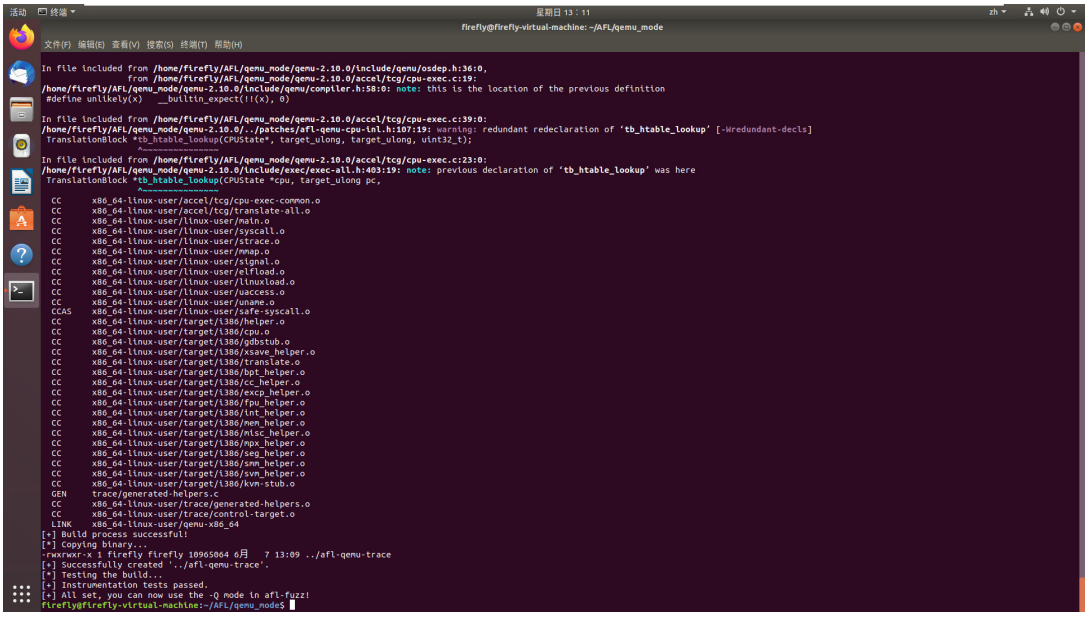


图 5：脚本输出 [+] Build process successful!成功提示

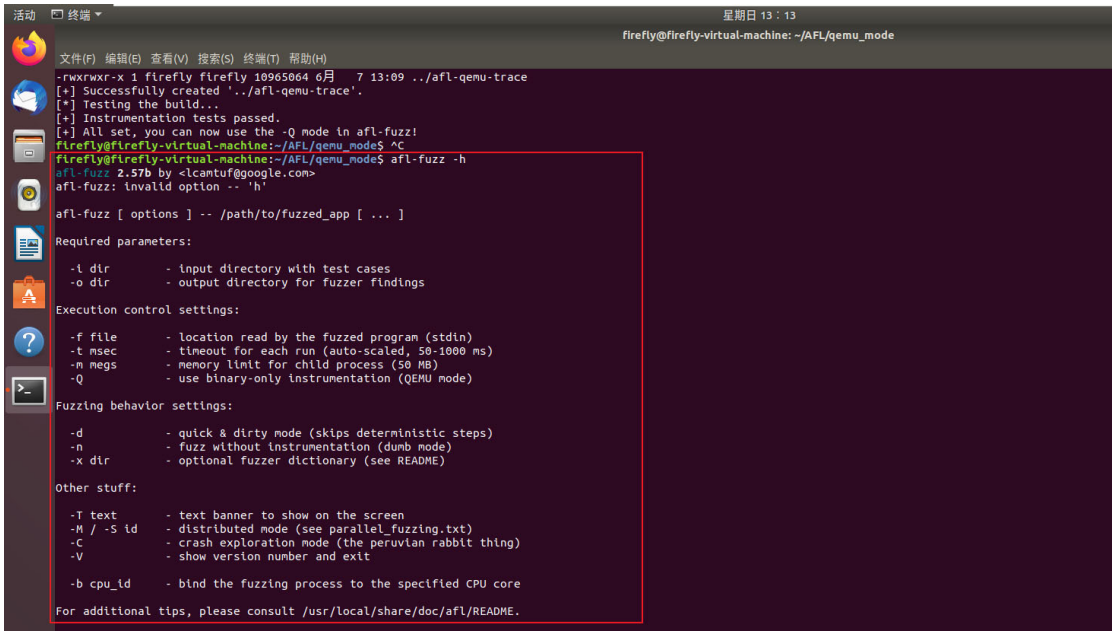


图 6：afl-fuzz -h 查看到 -Q (QEMU mode) 选项，代表动态插桩功能就绪

3.4 下载并解压 Coreutils-9.1

```bash
cd ~
wget https://ftp.gnu.org/gnu/coreutils/coreutils-9.1.tar.gz
tar -zxvf coreutils-9.1.tar.gz
cd coreutils-9.1
```

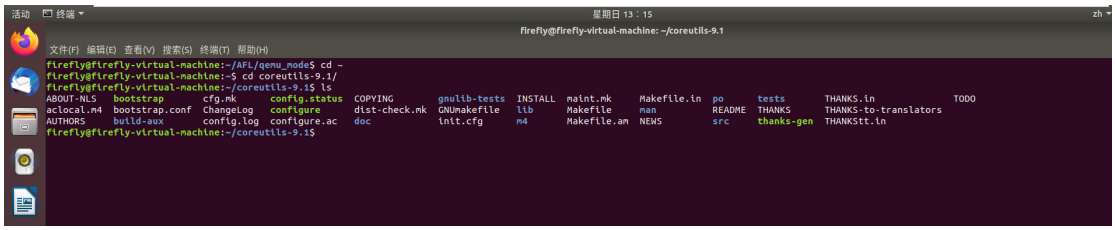


图 7：下载、解压 coreutils 源码包后的文件展示终端界面

3.5 修复 AFL 运行核心配置

AFL 对系统内核崩溃转储配置有要求，执行以下命令临时修改配置，规避报错：

```bash
sudo su
echo core > /proc/sys/kernel/core_pattern
exit
```

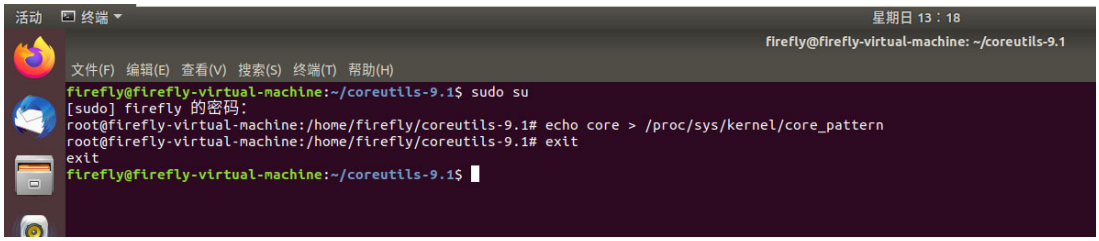


图 8：修改 core_pattern 配置的执行界面

4. 实验一：基于 afl-gcc 静态编译插桩模糊测试

4.1 使用 afl-gcc 编译 Coreutils

在 coreutils 根目录执行配置命令，指定编译器为 afl-gcc，并编译源码：

```
bash
./configure CC=afl-gcc
make
cd src
```

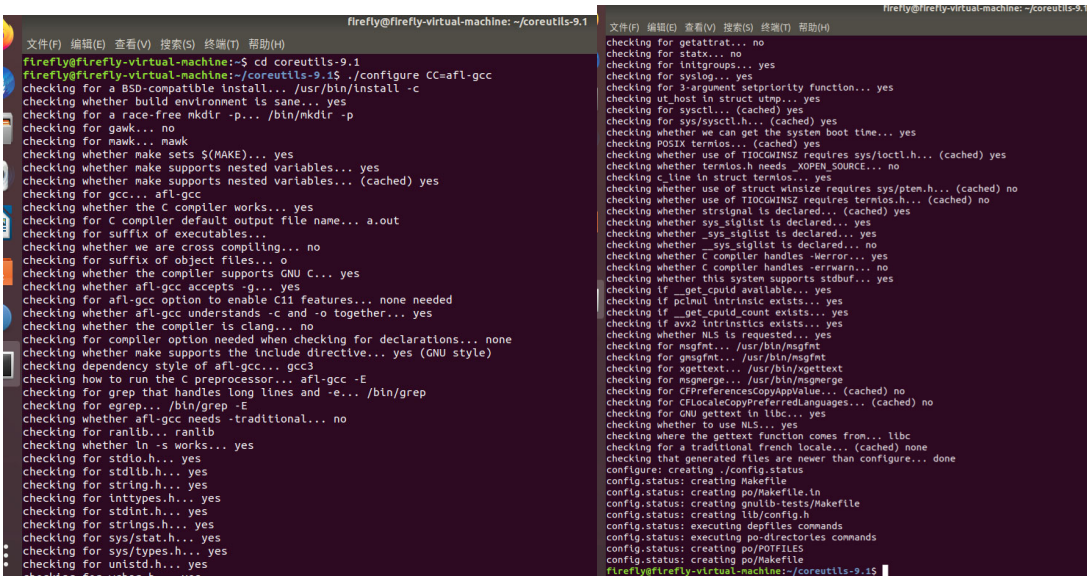


图 9：./configure CC=afl-gcc 配置执行界面

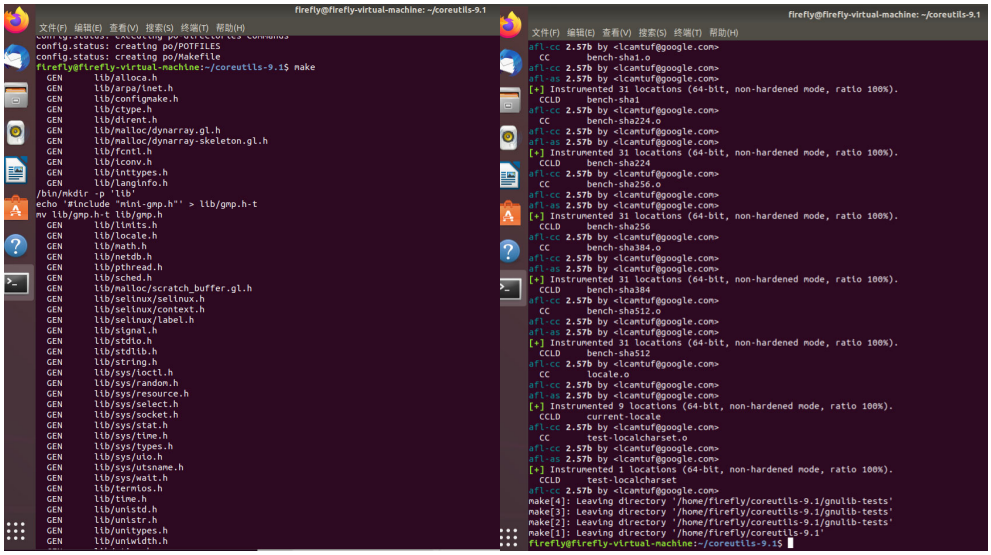


图 10：make 编译 coreutils 过程及结束界面

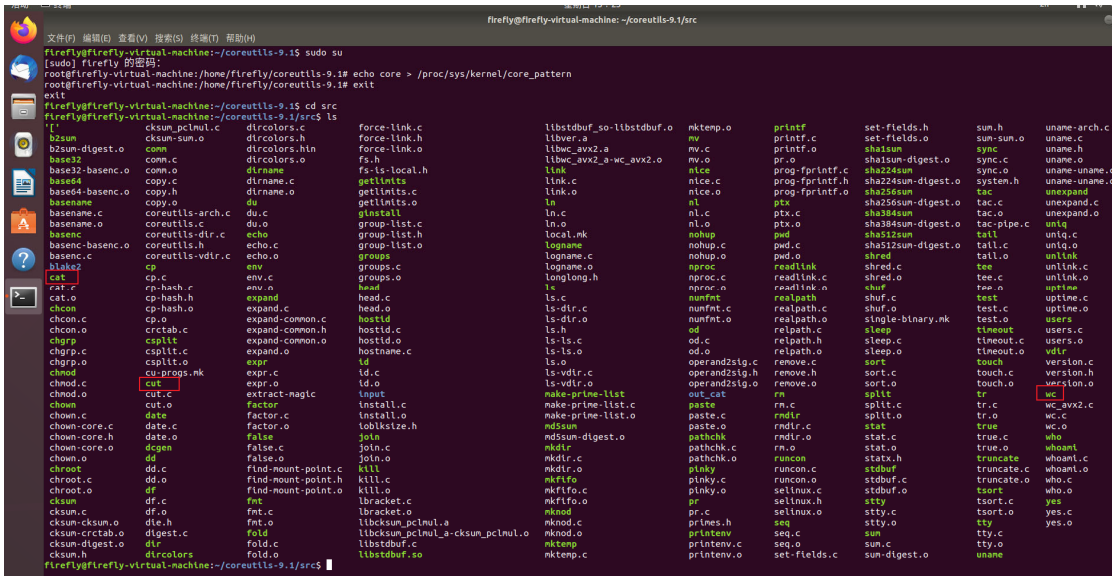


图 11：cd src 后执行 ls，查看 cat、wc、cut 程序文件

4.2 创建输入种子目录并构造测试用例

创建种子文件夹，分别构造**合法语义种子**与**随机畸形种子**：

```bash
mkdir input
cd input
# 合法语义种子
echo "hello world" > seed1.txt
echo 'test 123 !@# ' > seed2.txt
echo "line1\nline2\nline3" > seed3.txt
# 随机畸形种子
echo -e "\x00\x01\xff@@####$$$" > seed_rand1.txt
echo "" > seed_empty.txt
```

```
cd ..
```



图 12：创建 input 目录及各类种子文件的执行界面

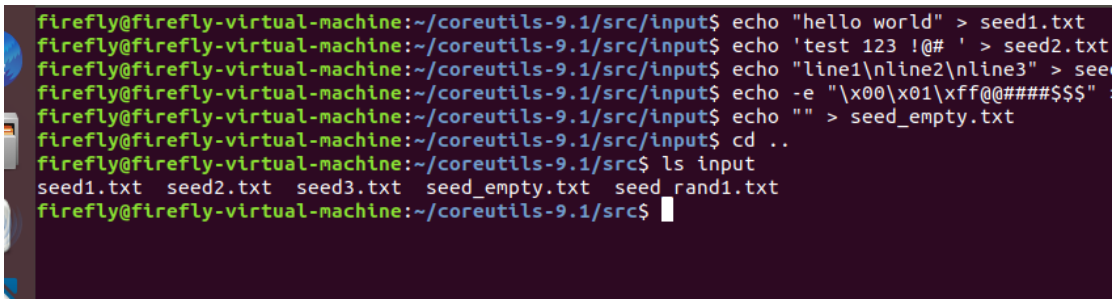


图 13：执行 ls input，查看目录内所有种子文件

4.3 启动静态插桩模糊测试

测试命令格式：`afl-fuzz -i 种子目录 -o 输出目录 ./程序名 @@`，其中 @@ 表示 AFL 将变异后的测试用例作为程序标准输入，每组测试运行一段时间后手动终止。

4.3.1 测试 cat 程序

```
bash
afl-fuzz -i input -o out_cat ./cat @@
```

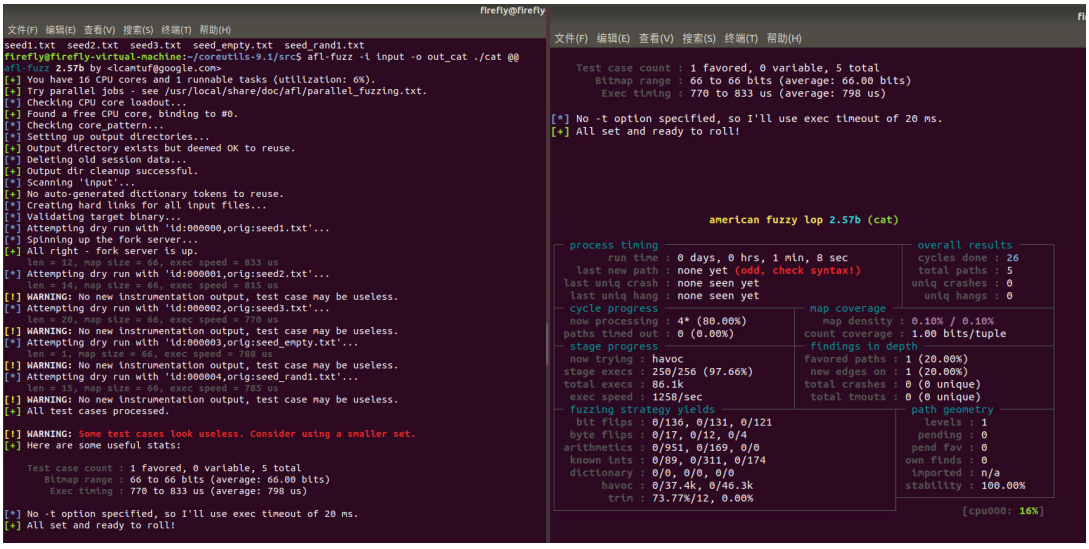


图 14：cat 程序静态插桩模式下 afl-fuzz 运行主界面

4.3.2 测试 wc 程序

```bash
afl-fuzz -i input -o out_wc ./wc @@
```

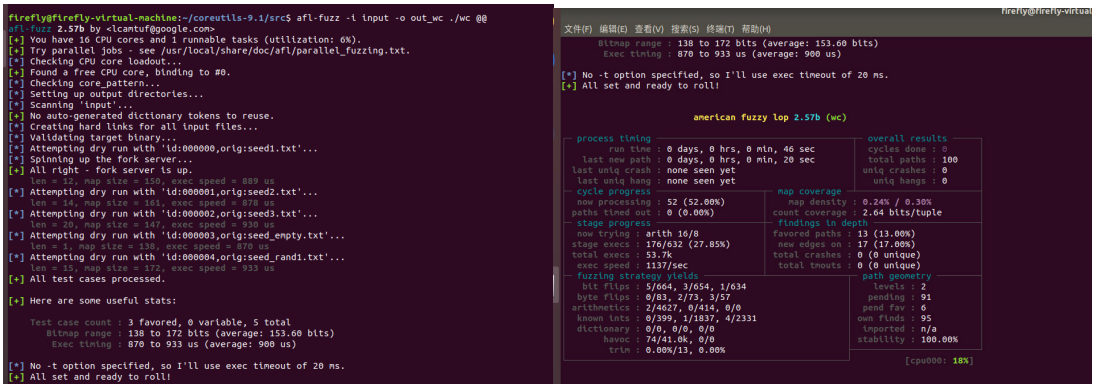


图 15：wc 程序静态插桩模式下 afl-fuzz 运行主界面

4.3.3 测试 cut 程序

```bash
afl-fuzz -i input -o out_cut ./cut @@
```

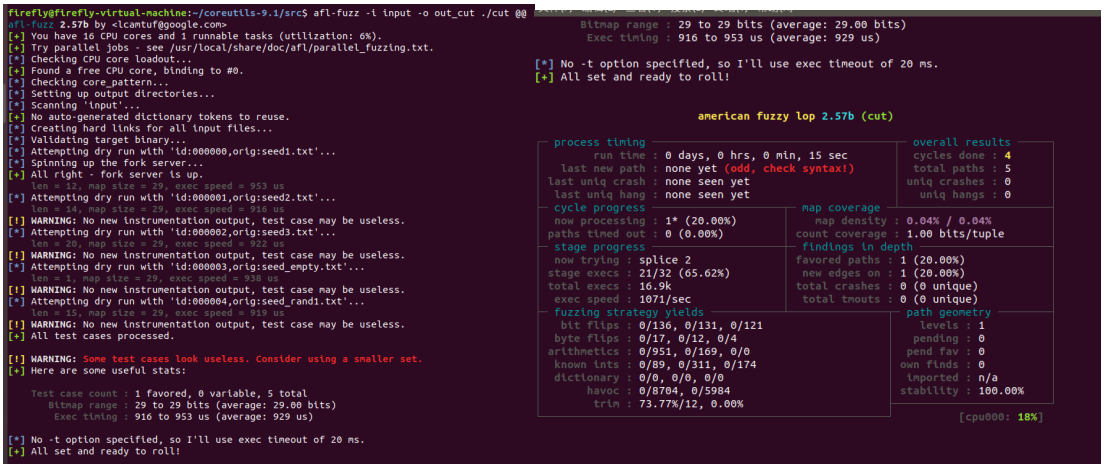


图 16：cut 程序静态插桩模式下 afl-fuzz 运行主界面

4.4 查看静态插桩测试结果

测试完成后，进入输出目录查看测试产物，核心文件包括：queue（新路径测试用例）、crashes（崩溃样本）、hangs（卡死样本）、fuzzer_stats（详细统计数据）。

```bash
cd out_cat
ls
cat fuzzer_stats
```

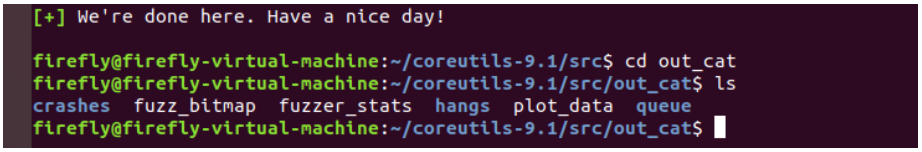
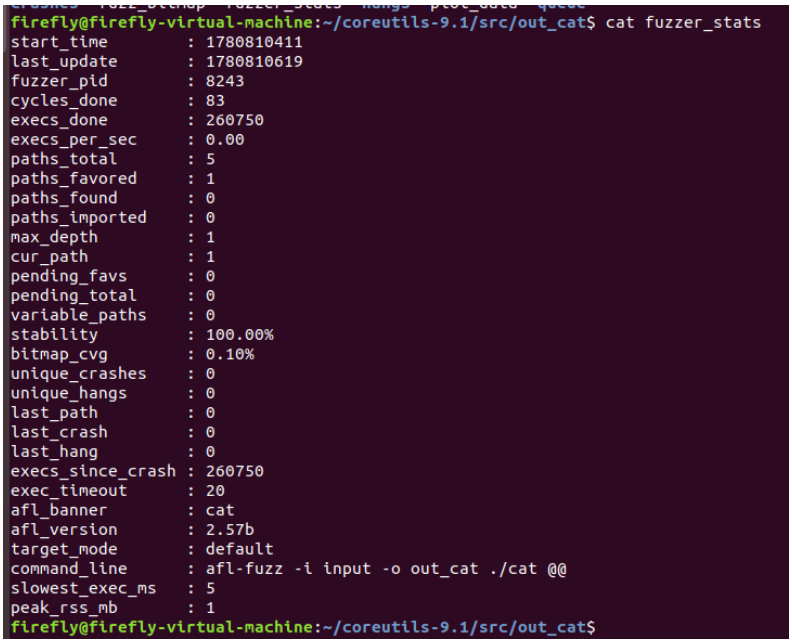


图 17：out_cat 目录下各类结果文件列表



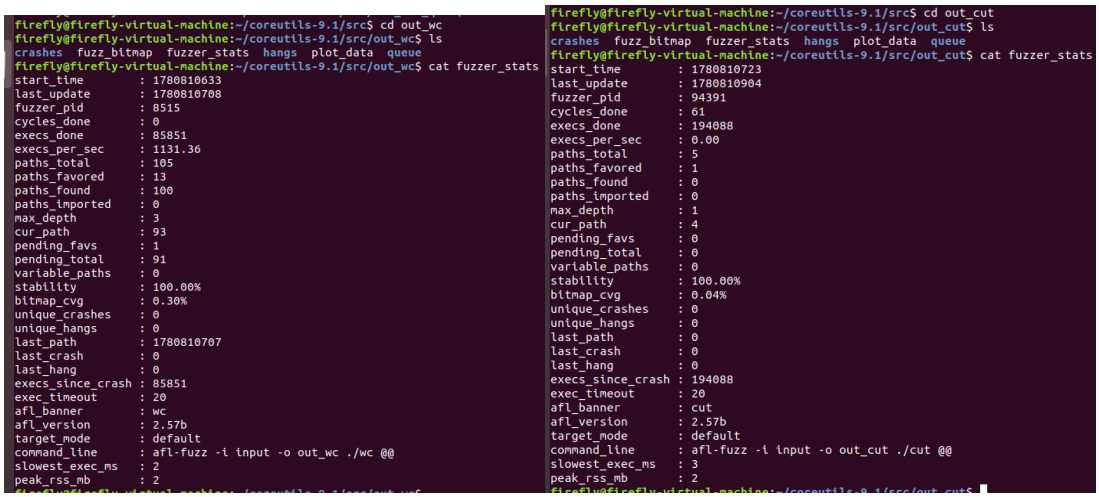


图 18：out_cat ,out_cut,out_wc 目录下 fuzzer_stats 统计文件完整内容

5. 实验二：基于 AFL-Qemu 动态插桩模糊测试

5.1 使用原生 GCC 重新编译 Coreutils

清理旧编译文件，使用系统默认 GCC 编译无插桩的原生二进制程序：

```bash
cd ~/coreutils-9.1
make clean
```

```
./configure
make
cd src
```



图 19：make clean 清理旧编译文件界面

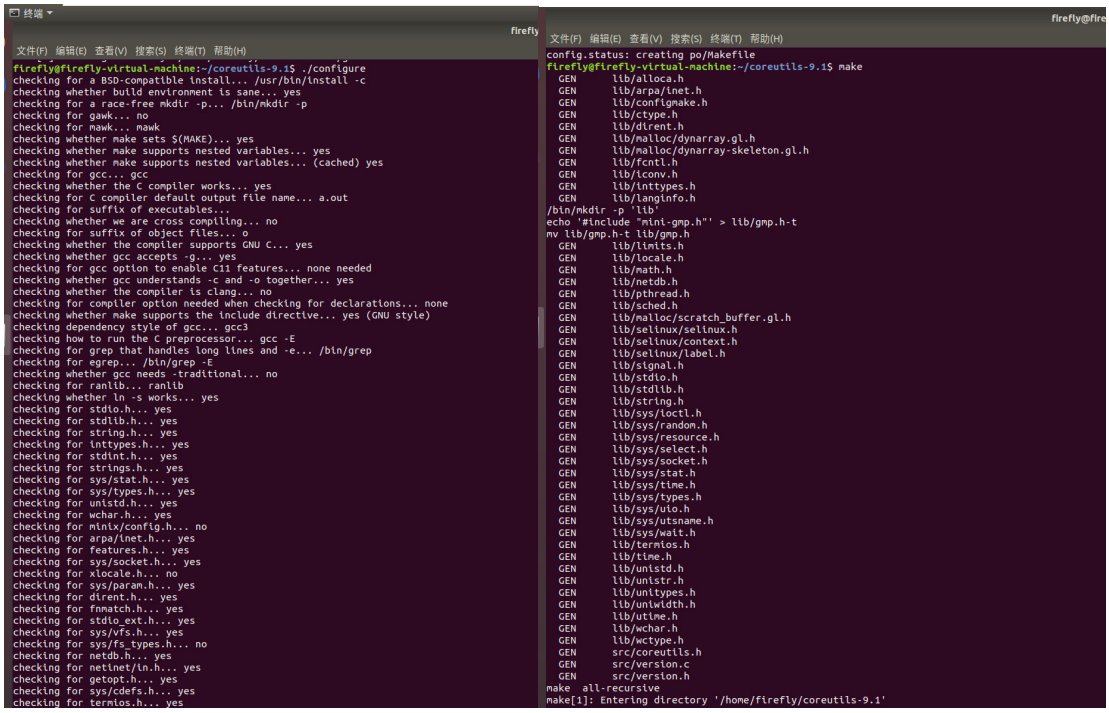


图 20：原生 GCC 执行 ./configure + make 编译界面

5.2 复用原有输入种子

直接沿用实验一 input 目录内的合法种子、畸形种子，无需重新创建，保证两组实验变量唯一、结果可对比。

5.3 启用 QEMU 模式执行模糊测试

在命令中添加 **-Q** 参数，开启 AFL-Qemu 动态二进制插桩模式，每组测试运行和之

前相同的一段时间。

5.3.1 QEMU 模式测试 cat

```
bash
afl-fuzz -Q -i input -o out_cat_qemu ./cat @@
```

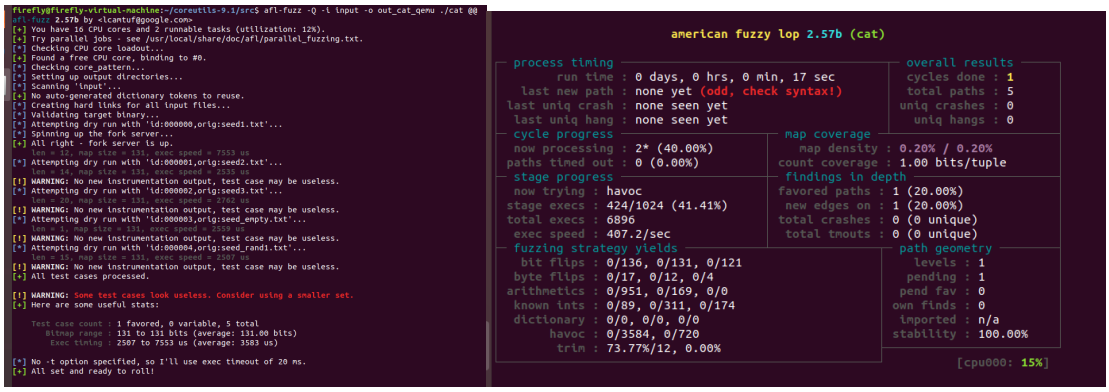


图 21：cat 程序 QEMU 动态插桩模式运行界面

5.3.2 QEMU 模式测试 wc

```
bash
afl-fuzz -Q -i input -o out_wc_qemu ./wc @@
```

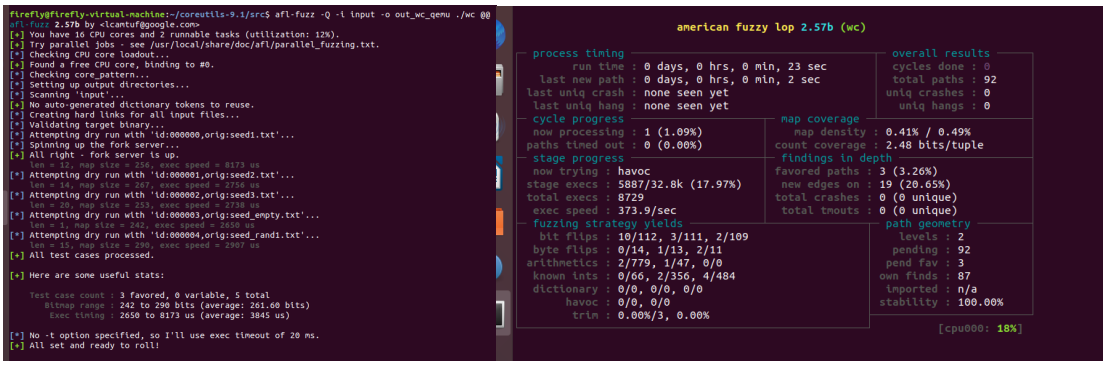


图 22：wc 程序 QEMU 动态插桩模式运行界面

5.3.3 QEMU 模式测试 cut

```
bash
afl-fuzz -Q -i input -o out_cut_qemu ./cut @@
```

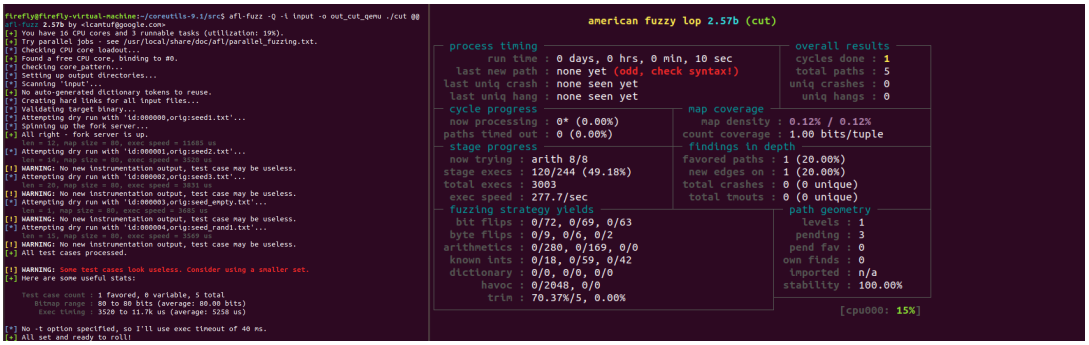


图 23：cut 程序 QEMU 动态插桩模式运行界面。

5.4 查看 QEMU 模式测试结果

进入 QEMU 对应输出目录，查看 queue、crashes、hangs、fuzzer_stats 等结果文件，对比静态插桩测试数据。

```
firefly@firefly-virtual-machine:~/coreutils-9.1/src$ cd out_wc_qemu
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_wc_qemu$ ls
crashes  fuzz_bitmap  fuzzer_stats  hangs  plot_data  queue
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_wc_qemu$ cat fuzzer_stats
start_time        : 1780811637
last_update       : 1780811706
fuzzer_pid        : 68299
cycles_done       : 0
execs_done        : 25608
execs_per_sec     : 0.00
paths_total       : 101
paths_favored     : 3
paths_found       : 96
paths_imported    : 0
max_depth         : 2
cur_path          : 1
pending_favs      : 3
pending_total     : 101
variable_paths    : 0
stability         : 100.00%
bitmap_cvg        : 0.49%
unique_crashes    : 0
unique_hangs      : 0
last_path         : 1780811689
last_crash        : 0
last_hang         : 0
execs_since_crash : 25608
exec_timeout      : 20
afl_banner        : wc
afl_version       : 2.57b
target_mode       : qemu
command_line      : afl-fuzz -Q -i input -o out_wc_qemu ./wc @@
slowest_exec_ms   : 47
peak_rss_mb       : 7
```

```
firefly@firefly-virtual-machine:~/coreutils-9.1/src$ cd out_cat_qemu
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_cat_qemu$ ls
crashes  fuzz_bitmap  fuzzer_stats  hangs  plot_data  queue
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_cat_qemu$ ^C
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_cat_qemu$ cat fuzzer_stats
start_time        : 1780811571
last_update       : 1780811630
fuzzer_pid        : 44047
cycles_done       : 7
execs_done        : 24228
execs_per_sec     : 0.00
paths_total       : 5
paths_favored     : 1
paths_found       : 0
paths_imported    : 0
max_depth         : 1
cur_path          : 2
pending_favs      : 0
pending_total     : 0
variable_paths    : 0
stability         : 100.00%
bitmap_cvg        : 0.20%
unique_crashes    : 0
unique_hangs      : 0
last_path         : 0
last_crash        : 0
last_hang         : 0
execs_since_crash : 24228
exec_timeout      : 20
afl_banner        : cat
afl_version       : 2.57b
target_mode       : qemu
command_line      : afl-fuzz -Q -i input -o out_cat_qemu ./cat @@
slowest_exec_ms   : 40
peak_rss_mb       : 7
```

```
firefly@firefly-virtual-machine:~/coreutils-9.1/src$ cd out_cut_qemu
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_cut_qemu$ ls
crashes  fuzz_bitmap  fuzzer_stats  hangs  plot_data  queue
firefly@firefly-virtual-machine:~/coreutils-9.1/src/out_cut_qemu$ cat fuzzer_stats
start_time        : 1780811713
last_update       : 1780811751
fuzzer_pid        : 93924
cycles_done       : 2
execs_done        : 10779
execs_per_sec     : 173.91
paths_total       : 5
paths_favored     : 1
paths_found       : 0
paths_imported    : 0
max_depth         : 1
cur_path          : 3
pending_favs      : 0
pending_total     : 0
variable_paths    : 0
stability         : 100.00%
bitmap_cvg        : 0.12%
unique_crashes    : 0
unique_hangs      : 0
last_path         : 0
last_crash        : 0
last_hang         : 0
execs_since_crash : 10779
exec_timeout      : 40
afl_banner        : cut
afl_version       : 2.57b
target_mode       : qemu
command_line      : afl-fuzz -Q -i input -o out_cut_qemu ./cut @@
slowest_exec_ms   : 68
peak_rss_mb       : 8
```

图 24：QEMU 模式输出目录及统计文件内容

6. 输入种子构造方式对 Fuzz 结果的影响分析

本次实验采用**合法语义种子**与**随机畸形种子**两组用例对照测试，种子构造方式对模糊测试的路径覆盖率、执行效率、漏洞挖掘效果均存在显著影响，具体分析如下：

6.1 对代码路径数量的影响

合法语义种子：输入符合程序设计逻辑，可顺利绕过前端输入校验，直接进入程序深层代码分支，最终探测到的代码路径总数更多，代码覆盖率更高。

随机畸形种子：特殊字符、空字节、乱码等异常输入大多会被程序输入校验逻辑拦截，难以进入程序内部分支，有效代码路径数量远少于合法种子。

6.2 对路径增长速度的影响

合法语义种子：测试前期路径增长速度较快，AFL 在合法输入基础上快速变异，持续探索相邻代码分支；测试中后期增长速度逐步放缓并趋于稳定，整体探索效率极高。

随机畸形种子：测试前期几乎没有新增路径，增长速度极慢；仅少量畸形输入侥幸绕过校验后，才会零星产生新路径，全程整体增速偏低。

6.3 对崩溃、卡死样本的影响

随机畸形种子更容易触发字符解析错误、内存越界、非法访问等异常问题，产出的 crashes、hangs 样本数量更多，擅长挖掘输入畸形类漏洞。合法种子变异后基本遵循程序正常执行流程，极少触发崩溃与卡死问题。

6.4 总结

合法输入种子优势是代码覆盖率高、路径探索效率强，适合全面检测程序常规逻辑隐患；随机畸形种子优势是易触发各类异常漏洞，但代码覆盖能力较差。在实际模糊测试工作中，一般会混合使用两类种子，兼顾代码覆盖率与漏洞挖掘能力，提升整体测试效果。

7. 静态编译插桩与 Qemu 动态插桩原理及效果差异分析

7.1 基本原理差异

7.1.1 afl-gcc 静态编译插桩

AFL 对 GCC 编译器进行封装改造，在源码编译阶段向二进制程序中嵌入桩代码，在每个代码基本块、分支跳转位置添加标记指令。程序运行时，内置桩代码实时反馈分支执行信息，AFL 以此判断是否发现新路径。该模式**必须获取程序源码，支持重新**

编译才可使用。

7.1.2 AFL-Qemu 动态二进制插桩

无需修改源码和编译流程，使用原生 GCC 编译程序。依靠 QEMU 模拟器动态翻译二进制指令，在指令实时翻译、模拟执行的过程中完成插桩与分支跟踪。该模式**不需要源码，可直接对闭源程序、固件、跨架构二进制文件开展测试**。

7.2 实验效果差异

结合本次同等时长对照测试，从四个核心维度对比两种模式：

1.执行效率：静态插桩无额外指令翻译开销，运行速度快；QEMU 存在指令模拟、动态翻译开销，执行速率约为静态插桩的 1/3 ~ 1/4。

2.路径探测能力：二者理论上可覆盖的代码分支范围一致；但静态插桩执行效率更高，相同测试时长内迭代次数更多，累计发现的新路径数量更多。

3.兼容性：静态插桩依赖源码与编译环境，仅适用于开源可编译程序；QEMU 动态插桩不依赖源码，适用场景更广。

4.稳定性：静态插桩后的程序行为和原生程序基本一致，稳定性高、误报少；QEMU 模拟环境偶尔存在兼容问题，可能产生误报的卡死、崩溃样本。

7.3 场景选型

拥有源码、可以重新编译程序时，优先使用 afl-gcc 静态插桩，追求高效、稳定的测试效果；仅拥有二进制文件、无源码（闭源程序、固件）时，使用 AFL-Qemu 动态插桩完成模糊测试。

8. 实验总结

本次实验完整完成了 AFL 工具安装、AFL-Qemu 动态插桩环境部署、Coreutils 工具集编译、静态与动态两种模式的模糊测试全流程实操，成功解决了 core_pattern 内核配置、环境兼容等常见报错，熟练掌握了 AFL 模糊测试工具的基础配置与使用方法。

通过对照实验明确了输入种子对模糊测试效果的核心影响：种子质量直接决定代码路径覆盖率、路径迭代速度和漏洞挖掘效率，合法种子保障测试广度，畸形种子提升漏洞挖掘概率，实际测试中需混合搭配使用。

同时深入掌握了 AFL 两种核心测试模式的原理与差异：静态编译插桩高效稳定、适配开源可编译程序，是源码测试的首选方案；QEMU 动态插桩无需源码、兼容性强，适配闭源二进制程序，但存在一定性能损耗。两种模式各司其职，可根据测试对象场景灵活选型。

本次测试对象 Coreutils 为工业级成熟工具集，代码健壮性较强，本次定时模糊测试未挖掘出高危崩溃类漏洞。通过本次实验，我彻底理解了二进制插桩与模糊测试的

核心原理，具备了基础的漏洞模糊测试实操与结果分析能力，为后续漏洞分析、安全测试相关学习奠定了扎实基础。